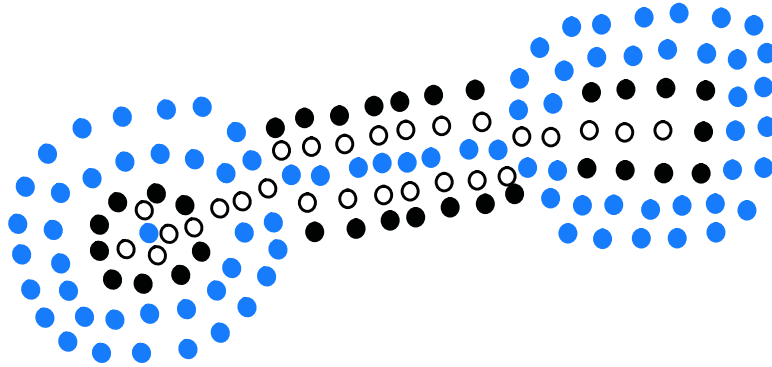




Contents

6.1	Stacks	226
6.1.1	The Stack Abstract Data Type	227
6.1.2	A Simple Array-Based Stack Implementation	230
6.1.3	Implementing a Stack with a Singly Linked List	233
6.1.4	Reversing an Array Using a Stack	234
6.1.5	Matching Parentheses and HTML Tags	235
6.2	Queues	238
6.2.1	The Queue Abstract Data Type	239
6.2.2	Array-Based Queue Implementation	241
6.2.3	Implementing a Queue with a Singly Linked List	245
6.2.4	A Circular Queue	246
6.3	Double-Ended Queues	248
6.3.1	The Deque Abstract Data Type	248
6.3.2	Implementing a Deque	250
6.3.3	Dequeues in the Java Collections Framework	251
6.4	Exercises	252

6.1 Stacks

A *stack* is a collection of objects that are inserted and removed according to the *last-in, first-out (LIFO)* principle. A user may insert objects into a stack at any time, but may only access or remove the most recently inserted object that remains (at the so-called “top” of the stack). The name “stack” is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser. In this case, the fundamental operations involve the “pushing” and “popping” of plates on the stack. When we need a new plate from the dispenser, we “pop” the top plate off the stack, and when we add a plate, we “push” it down on the stack to become the new top plate. Perhaps an even more amusing example is a PEZ[®] candy dispenser, which stores mint candies in a spring-loaded container that “pops” out the topmost candy in the stack when the top of the dispenser is lifted (see Figure 6.1).

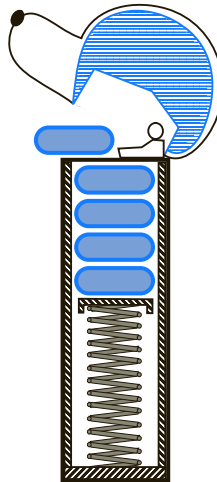


Figure 6.1: A schematic drawing of a PEZ[®] dispenser; a physical implementation of the stack ADT. (PEZ[®] is a registered trademark of PEZ Candy, Inc.)

Stacks are a fundamental data structure. They are used in many applications, including the following.

Example 6.1: Internet Web browsers store the addresses of recently visited sites on a stack. Each time a user visits a new site, that site’s address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.

Example 6.2: Text editors usually provide an “undo” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.

6.1.1 The Stack Abstract Data Type

Stacks are the simplest of all data structures, yet they are also among the most important, as they are used in a host of different applications, and as a tool for many more sophisticated data structures and algorithms. Formally, a stack is an abstract data type (ADT) that supports the following two update methods:

push(*e*): Adds element *e* to the top of the stack.

pop(): Removes and returns the top element from the stack (or null if the stack is empty).

Additionally, a stack supports the following accessor methods for convenience:

top(): Returns the top element of the stack, without removing it (or null if the stack is empty).

size(): Returns the number of elements in the stack.

isEmpty(): Returns a boolean indicating whether the stack is empty.

By convention, we assume that elements added to the stack can have arbitrary type and that a newly created stack is empty.

Example 6.3: *The following table shows a series of stack operations and their effects on an initially empty stack *S* of integers.*

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

A Stack Interface in Java

In order to formalize our abstraction of a stack, we define what is known as its **application programming interface** (API) in the form of a Java **interface**, which describes the names of the methods that the ADT supports and how they are to be declared and used. This interface is defined in Code Fragment 6.1.

We rely on Java’s **generics framework** (described in Section 2.5.2), allowing the elements stored in the stack to belong to any object type $\langle E \rangle$. For example, a variable representing a stack of integers could be declared with type `Stack<Integer>`. The formal type parameter is used as the parameter type for the push method, and the return type for both pop and top.

Recall, from the discussion of Java interfaces in Section 2.3.1, that the interface serves as a type definition but that it cannot be directly instantiated. For the ADT to be of any use, we must provide one or more concrete classes that implement the methods of the interface associated with that ADT. In the following subsections, we will give two such implementations of the Stack interface: one that uses an array for storage and another that uses a linked list.

The java.util.Stack Class

Because of the importance of the stack ADT, Java has included, since its original version, a concrete class named `java.util.Stack` that implements the LIFO semantics of a stack. However, Java’s `Stack` class remains only for historic reasons, and its interface is not consistent with most other data structures in the Java library. In fact, the current documentation for the `Stack` class recommends that it not be used, as LIFO functionality (and more) is provided by a more general data structure known as a double-ended queue (which we describe in Section 6.3).

For the sake of comparison, Table 6.1 provides a side-by-side comparison of the interface for our stack ADT and the `java.util.Stack` class. In addition to some differences in method names, we note that methods `pop` and `peek` of the `java.util.Stack` class throw a custom `EmptyStackException` if called when the stack is empty (whereas `null` is returned in our abstraction).

Our Stack ADT	Class <code>java.util.Stack</code>	
<code>size()</code>	<code>size()</code>	
<code>isEmpty()</code>	<code>empty()</code>	⇐
<code>push(<i>e</i>)</code>	<code>push(<i>e</i>)</code>	
<code>pop()</code>	<code>pop()</code>	
<code>top()</code>	<code>peek()</code>	⇐

Table 6.1: Methods of our stack ADT and corresponding methods of the class `java.util.Stack`, with differences highlighted in the right margin.

```
1  /**
2   * A collection of objects that are inserted and removed according to the last-in
3   * first-out principle. Although similar in purpose, this interface differs from
4   * java.util.Stack.
5   *
6   * @author Michael T. Goodrich
7   * @author Roberto Tamassia
8   * @author Michael H. Goldwasser
9   */
10 public interface Stack<E> {
11
12     /**
13      * Returns the number of elements in the stack.
14      * @return number of elements in the stack
15      */
16     int size();
17
18     /**
19      * Tests whether the stack is empty.
20      * @return true if the stack is empty, false otherwise
21      */
22     boolean isEmpty();
23
24     /**
25      * Inserts an element at the top of the stack.
26      * @param e the element to be inserted
27      */
28     void push(E e);
29
30     /**
31      * Returns, but does not remove, the element at the top of the stack.
32      * @return top element in the stack (or null if empty)
33      */
34     E top();
35
36     /**
37      * Removes and returns the top element from the stack.
38      * @return element removed (or null if empty)
39      */
40     E pop();
41 }
```

Code Fragment 6.1: Interface Stack documented with comments in Javadoc style (Section 1.9.4). Note also the use of the generic parameterized type, E, which allows a stack to contain elements of any specified (reference) type.

6.1.2 A Simple Array-Based Stack Implementation

As our first implementation of the stack ADT, we store elements in an array, named `data`, with capacity N for some fixed N . We oriented the stack so that the bottom element of the stack is always stored in cell `data[0]`, and the top element of the stack in cell `data[t]` for index t that is equal to one less than the current size of the stack. (See Figure 6.2.)

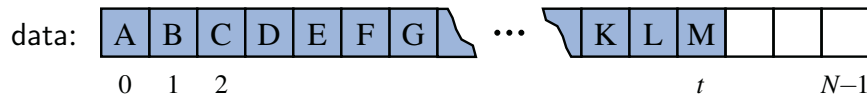


Figure 6.2: Representing a stack with an array; the top element is in cell `data[t]`.

Recalling that arrays start at index 0 in Java, when the stack holds elements from `data[0]` to `data[t]` inclusive, it has size $t + 1$. By convention, when the stack is empty it will have t equal to -1 (and thus has size $t + 1$, which is 0). A complete Java implementation based on this strategy is given in Code Fragment 6.2 (with Javadoc comments omitted due to space considerations).

```

1 public class ArrayStack<E> implements Stack<E> {
2     public static final int CAPACITY=1000; // default array capacity
3     private E[] data; // generic array used for storage
4     private int t = -1; // index of the top element in stack
5     public ArrayStack() { this(CAPACITY); } // constructs stack with default capacity
6     public ArrayStack(int capacity) { // constructs stack with given capacity
7         data = (E[]) new Object[capacity]; // safe cast; compiler may give warning
8     }
9     public int size() { return (t + 1); }
10    public boolean isEmpty() { return (t == -1); }
11    public void push(E e) throws IllegalStateException {
12        if (size() == data.length) throw new IllegalStateException("Stack is full");
13        data[++t] = e; // increment t before storing new item
14    }
15    public E top() {
16        if (isEmpty()) return null;
17        return data[t];
18    }
19    public E pop() {
20        if (isEmpty()) return null;
21        E answer = data[t];
22        data[t] = null; // dereference to help garbage collection
23        t--;
24        return answer;
25    }
26 }

```

Code Fragment 6.2: Array-based implementation of the Stack interface.

A Drawback of This Array-Based Stack Implementation

The array implementation of a stack is simple and efficient. Nevertheless, this implementation has one negative aspect—it relies on a fixed-capacity array, which limits the ultimate size of the stack.

For convenience, we allow the user of a stack to specify the capacity as a parameter to the constructor (and offer a default constructor that uses capacity of 1,000). In cases where a user has a good estimate on the number of items needing to go in the stack, the array-based implementation is hard to beat. However, if the estimate is wrong, there can be grave consequences. If the application needs much less space than the reserved capacity, memory is wasted. Worse yet, if an attempt is made to push an item onto a stack that has already reached its maximum capacity, the implementation of Code Fragment 6.2 throws an `IllegalStateException`, refusing to store the new element. Thus, even with its simplicity and efficiency, the array-based stack implementation is not necessarily ideal.

Fortunately, we will later demonstrate two approaches for implementing a stack without such a size limitation and with space always proportional to the actual number of elements stored in the stack. One approach, given in the next subsection uses a singly linked list for storage; in Section 7.2.1, we will provide a more advanced array-based approach that overcomes the limit of a fixed capacity.

Analyzing the Array-Based Stack Implementation

The correctness of the methods in the array-based implementation follows from our definition of index t . Note well that when pushing an element, t is incremented before placing the new element, so that it uses the first available cell.

Table 6.2 shows the running times for methods of this array-based stack implementation. Each method executes a constant number of statements involving arithmetic operations, comparisons, and assignments, or calls to `size` and `isEmpty`, which both run in constant time. Thus, in this implementation of the stack ADT, each method runs in constant time, that is, they each run in $O(1)$ time.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
top	$O(1)$
push	$O(1)$
pop	$O(1)$

Table 6.2: Performance of a stack realized by an array. The space usage is $O(N)$, where N is the size of the array, determined at the time the stack is instantiated, and independent from the number $n \leq N$ of elements that are actually in the stack.

Garbage Collection in Java

We wish to draw attention to one interesting aspect involving the implementation of the pop method in Code Fragment 6.2. We set a local variable, *answer*, to reference the element that is being popped, and then we intentionally reset *data[t]* to **null** at line 22, before decrementing *t*. The assignment to **null** was not technically required, as our stack would still operate correctly without it.

Our reason for returning the cell to a null reference is to assist Java's *garbage collection* mechanism, which searches memory for objects that are no longer actively referenced and reclaims their space for future use. (For more details, see Section 15.1.3.) If we continued to store a reference to the popped element in our array, the stack class would ignore it (eventually overwriting the reference if more elements get added to the stack). But, if there were no other active references to the element in the user's application, that spurious reference in the stack's array would stop Java's garbage collector from reclaiming the element.

Sample Usage

We conclude this section by providing a demonstration of code that creates and uses an instance of the `ArrayStack` class. In this example, we declare the parameterized type of the stack as the `Integer` wrapper class. This causes the signature of the push method to accept an `Integer` instance as a parameter, and for the return type of both `top` and `pop` to be an `Integer`. Of course, with Java's autoboxing and unboxing (see Section 1.3), a primitive **int** can be sent as a parameter to `push`.

```
Stack<Integer> S = new ArrayStack<>(); // contents: ()
S.push(5); // contents: (5)
S.push(3); // contents: (5, 3)
System.out.println(S.size()); // contents: (5, 3) outputs 2
System.out.println(S.pop()); // contents: (5) outputs 3
System.out.println(S.isEmpty()); // contents: (5) outputs false
System.out.println(S.pop()); // contents: () outputs 5
System.out.println(S.isEmpty()); // contents: () outputs true
System.out.println(S.pop()); // contents: () outputs null
S.push(7); // contents: (7)
S.push(9); // contents: (7, 9)
System.out.println(S.top()); // contents: (7, 9) outputs 9
S.push(4); // contents: (7, 9, 4)
System.out.println(S.size()); // contents: (7, 9, 4) outputs 3
System.out.println(S.pop()); // contents: (7, 9) outputs 4
S.push(6); // contents: (7, 9, 6)
S.push(8); // contents: (7, 9, 6, 8)
System.out.println(S.pop()); // contents: (7, 9, 6) outputs 8
```

Code Fragment 6.3: Sample usage of our `ArrayStack` class.

6.1.3 Implementing a Stack with a Singly Linked List

In this section, we demonstrate how the Stack interface can be easily implemented using a singly linked list for storage. Unlike our array-based implementation, the linked-list approach has memory usage that is always proportional to the number of actual elements currently in the stack, and without an arbitrary capacity limit.

In designing such an implementation, we need to decide if the top of the stack is at the front or back of the list. There is clearly a best choice here, however, since we can insert and delete elements in constant time only at the front. With the top of the stack stored at the front of the list, all methods execute in constant time.

The Adapter Pattern

The *adapter* design pattern applies to any context where we effectively want to modify an existing class so that its methods match those of a related, but different, class or interface. One general way to apply the adapter pattern is to define a new class in such a way that it contains an instance of the existing class as a hidden field, and then to implement each method of the new class using methods of this hidden instance variable. By applying the adapter pattern in this way, we have created a new class that performs some of the same functions as an existing class, but repackaged in a more convenient way.

In the context of the stack ADT, we can adapt our SinglyLinkedList class of Section 3.2.1 to define a new LinkedStack class, shown in Code Fragment 6.4. This class declares a SinglyLinkedList named list as a private field, and uses the following correspondences:

<i>Stack Method</i>	<i>Singly Linked List Method</i>
size()	list.size()
isEmpty()	list.isEmpty()
push(<i>e</i>)	list.addFirst(<i>e</i>)
pop()	list.removeFirst()
top()	list.first()

```

1 public class LinkedStack<E> implements Stack<E> {
2     private SinglyLinkedList<E> list = new SinglyLinkedList<>(); // an empty list
3     public LinkedStack() { } // new stack relies on the initially empty list
4     public int size() { return list.size(); }
5     public boolean isEmpty() { return list.isEmpty(); }
6     public void push(E element) { list.addFirst(element); }
7     public E top() { return list.first(); }
8     public E pop() { return list.removeFirst(); }
9 }

```

Code Fragment 6.4: Implementation of a Stack using a SinglyLinkedList as storage.

6.1.4 Reversing an Array Using a Stack

As a consequence of the LIFO protocol, a stack can be used as a general toll to reverse a data sequence. For example, if the values 1, 2, and 3 are pushed onto a stack in that order, they will be popped from the stack in the order 3, 2, and then 1.

We demonstrate this concept by revisiting the problem of reversing the elements of an array. (We provided a recursive algorithm for this task in Section 5.3.1.) We create an empty stack for auxiliary storage, push all of the array elements onto the stack, and then pop those elements off of the stack while overwriting the cells of the array from beginning to end. In Code Fragment 6.5, we give a Java implementation of this algorithm. We show an example use of this method in Code Fragment 6.6.

```

1  /** A generic method for reversing an array. */
2  public static <E> void reverse(E[] a) {
3      Stack<E> buffer = new ArrayStack<>(a.length);
4      for (int i=0; i < a.length; i++)
5          buffer.push(a[i]);
6      for (int i=0; i < a.length; i++)
7          a[i] = buffer.pop();
8  }
```

Code Fragment 6.5: A generic method that reverses the elements in an array with objects of type E, using a stack declared with the interface Stack<E> as its type.

```

1  /** Tester routine for reversing arrays */
2  public static void main(String args[] ) {
3      Integer[] a = {4, 8, 15, 16, 23, 42}; // autoboxing allows this
4      String[] s = {"Jack", "Kate", "Hurley", "Jin", "Michael"};
5      System.out.println("a = " + Arrays.toString(a));
6      System.out.println("s = " + Arrays.toString(s));
7      System.out.println("Reversing...");
8      reverse(a);
9      reverse(s);
10     System.out.println("a = " + Arrays.toString(a));
11     System.out.println("s = " + Arrays.toString(s));
12 }
```

The output from this method is the following:

```

a = [4, 8, 15, 16, 23, 42]
s = [Jack, Kate, Hurley, Jin, Michael]
Reversing...
a = [42, 23, 16, 15, 8, 4]
s = [Michael, Jin, Hurley, Kate, Jack]
```

Code Fragment 6.6: A test of the reverse method using two arrays.

6.1.5 Matching Parentheses and HTML Tags

In this subsection, we explore two related applications of stacks, both of which involve testing for pairs of matching delimiters. In our first application, we consider arithmetic expressions that may contain various pairs of grouping symbols, such as

- Parentheses: “(” and “)”
- Braces: “{” and “}”
- Brackets: “[” and “]”

Each opening symbol must match its corresponding closing symbol. For example, a left bracket, “[,” must match a corresponding right bracket, “],” as in the following expression

$$[(5 + x) - (y + z)].$$

The following examples further illustrate this concept:

- Correct: `()(()){([()] )}`
- Correct: `((())(()){([()] )})`
- Incorrect: `)((()){([()] )}`
- Incorrect: `{([()] )}`
- Incorrect: `(`

We leave the precise definition of a matching group of symbols to Exercise R-6.6.

An Algorithm for Matching Delimiters

An important task when processing arithmetic expressions is to make sure their delimiting symbols match up correctly. We can use a stack to perform this task with a single left-to-right scan of the original string.

Each time we encounter an opening symbol, we push that symbol onto the stack, and each time we encounter a closing symbol, we pop a symbol from the stack (assuming it is not empty) and check that these two symbols form a valid pair. If we reach the end of the expression and the stack is empty, then the original expression was properly matched. Otherwise, there must be an opening delimiter on the stack without a matching symbol. If the length of the original expression is n , the algorithm will make at most n calls to push and n calls to pop. Code Fragment 6.7 presents a Java implementation of such an algorithm. It specifically checks for delimiter pairs `()`, `{ }`, and `[ ]`, but could easily be changed to accommodate further symbols. Specifically, we define two fixed strings, `"( { [ "` and `) } ] "`, that are intentionally coordinated to reflect the symbol pairs. When examining a character of the expression string, we call the `indexOf` method of the `String` class on these special strings to determine if the character matches a delimiter and, if so, which one. Method `indexOf` returns the index at which a given character is first found in a string (or `-1` if the character is not found).

```

1  /** Tests if delimiters in the given expression are properly matched. */
2  public static boolean isMatched(String expression) {
3      final String opening  = "{[";           // opening delimiters
4      final String closing  = "}]";           // respective closing delimiters
5      Stack<Character> buffer = new LinkedStack<>();
6      for (char c : expression.toCharArray()) {
7          if (opening.indexOf(c) != -1)        // this is a left delimiter
8              buffer.push(c);
9          else if (closing.indexOf(c) != -1) {  // this is a right delimiter
10             if (buffer.isEmpty())             // nothing to match with
11                 return false;
12             if (closing.indexOf(c) != opening.indexOf(buffer.pop()))
13                 return false;                // mismatched delimiter
14         }
15     }
16     return buffer.isEmpty();                 // were all opening delimiters matched?
17 }

```

Code Fragment 6.7: Method for matching delimiters in an arithmetic expression.

Matching Tags in a Markup Language

Another application of matching delimiters is in the validation of markup languages such as HTML or XML. HTML is the standard format for hyperlinked documents on the Internet and XML is an extensible markup language used for a variety of structured data sets. We show a sample HTML document in Figure 6.3.

```

<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>

```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Figure 6.3: Illustrating (a) an HTML document and (b) its rendering.

In an HTML document, portions of text are delimited by *HTML tags*. A simple opening HTML tag has the form “<name>” and the corresponding closing tag has the form “</name>”. For example, we see the <body> tag on the first line of Figure 6.3a, and the matching </body> tag at the close of that document. Other commonly used HTML tags that are used in this example include:

- <body>: document body
- <h1>: section header
- <center>: center justify
- <p>: paragraph
- : numbered (ordered) list
- : list item

Ideally, an HTML document should have matching tags, although most browsers tolerate a certain number of mismatching tags. In Code Fragment 6.8, we give a Java method that matches tags in a string representing an HTML document.

We make a left-to-right pass through the raw string, using index *j* to track our progress. The indexOf method of the String class, which optionally accepts a starting index as a second parameter, locates the '<' and '>' characters that define the tags. Method substring, also of the String class, returns the substring starting at a given index and optionally ending right before another given index. Opening tags are pushed onto the stack, and matched against closing tags as they are popped from the stack, just as we did when matching delimiters in Code Fragment 6.7.

```

1  /** Tests if every opening tag has a matching closing tag in HTML string. */
2  public static boolean isHTMLMatched(String html) {
3      Stack<String> buffer = new LinkedStack<>();
4      int j = html.indexOf('<');           // find first '<' character (if any)
5      while (j != -1) {
6          int k = html.indexOf('>', j+1); // find next '>' character
7          if (k == -1)
8              return false;             // invalid tag
9          String tag = html.substring(j+1, k); // strip away < >
10         if (!tag.startsWith("/"))        // this is an opening tag
11             buffer.push(tag);
12         else {                            // this is a closing tag
13             if (buffer.isEmpty())
14                 return false;           // no tag to match
15             if (!tag.substring(1).equals(buffer.pop()))
16                 return false;          // mismatched tag
17         }
18         j = html.indexOf('<', k+1);       // find next '<' character (if any)
19     }
20     return buffer.isEmpty();             // were all opening tags matched?
21 }
```

Code Fragment 6.8: Method for testing if an HTML document has matching tags.